Openi nCol ab

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, cal
l drive.mount("/content/drive", force_remount=True).
```

In [ ]:
```
import pandas as pd
import numpy as np
import sklearn as sk
from sklearn.preprocessing import StandardScaler
```

In [ ]:
```
df = pd.read_csv('/content/wireless_signal_quality_dataset.csv')
print(df.isnull().sum())
```

```
Signal_Strength_dBm        0
SNR_dB                     0
Interference_dB            0
Distance_m                 0
Frequency_GHz              0
Channel_Bandwidth_MHz      0
Noise_Floor_dBm            0
Connected_Devices          0
Estimated_Throughput_Mbps  0
Quality                    0
dtype: int64
```

In [ ]:
```
file = "/content/wireless_signal_quality_dataset.csv"

data = pd.read_csv(file)
```

In [ ]:
```
print(data.head())
```

```
   Signal_Strength_dBm  SNR_dB  Interference_dB  Distance_m  Frequency_GHz
\
0                  -41       6                0         257            5.0
1                  -86      25               20         134            2.4
2                  -73      27                0         385            2.4
3                  -57       5                0         255            5.0
4                  -87      15               24          61            5.0

   Channel_Bandwidth_MHz  Noise_Floor_dBm  Connected_Devices \
0                    160             -100                 24
1                    160              -96                 41
2                     20              -72                 13
3                    160              -86                  9
4                     20              -79                 32

   Estimated_Throughput_Mbps Quality
0                         18    Poor
1                         55    Poor
2                         81    Fair
3                         15    Poor
4                         21    Poor
```

In [ ]:
```
print(data.tail())
```

```
    Signal_Strength_dBm SNR_dB Interference_dB Distance_m Frequency_GHz
\
95                  -75      37              11         17           2.4
96                  -88      11              18        231           5.0
97                  -57       6              27        460           5.0
98                  -49      25              24        298           2.4
99                  -46       8              29        423           5.0

    Channel_Bandwidth_MHz Noise_Floor_dBm Connected_Devices \
95                    160            -80                34
96                     20            -90                25
97                    160            -76                24
98                     20            -78                39
99                     40            -71                16

    Estimated_Throughput_Mbps Quality
95                        100    Fair
96                         15    Poor
97                          1    Poor
98                         51    Fair
99                          1    Poor
```

In [ ]:
```
label_encoder = LabelEncoder()
data['Quality'] = label_encoder.fit_transform(data['Quality'])
```

In [ ]:
```
X = data.drop('Quality', axis=1)
y = data['Quality']
```

In [ ]:
```
X = pd.get_dummies(X, drop_first=True)
```

In [ ]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
```

In [ ]:
```
#Train test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,  test_size=0.2,
    random_state=
    42
    )
```

In [ ]:
```
# Feature scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [ ]:
```
#Train Logistic Regression Model
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

Out[ ]:
```
LogisticRegression(max_iter=1000)
```
InaJupyterenvironment,pleasererunthiscell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [ ]:
```
#Makepredictions on thetestset
y_pred = model.predict(X_test)
```

In [ ]:
```
# Evaluation
print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))
```

```
Accuracy:
0.65
```

In [ ]:
```
print("\nAccuracy:")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy:
[[0 0 1 0]
[0 4 0 2]
[0 2 1 0]
[0 2 0 8]]
```

In [ ]:
```
print(" Classification Report: ")
print(classification_report(y_test, y_pred))
```

```
    Classification Report:
                  precision    recall f1-score       support

               0       0.00      0.00      0.00             1
               1       0.50      0.67      0.57             6
               2       0.50      0.33      0.40             3
               3       0.80      0.80      0.80            10

        accuracy                           0.65             2
       macro avg       0.45      0.45      0.44             0
    weighted avg       0.62      0.65      0.63             2
                                                            0
                                                            2
                                                            0
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1      565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0i n labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1      565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 i
```